

1. (a) The 2-dimensional array can be defined as an arrays. The two-dimensional arrays are organized as matrix which can be represented as the collection of rows and column as array $[M][N]$ Where M is the number of rows and N is the number of column. To find the address of any element in a 2-Dimensional array are the following ways -

1. Row Major order.
2. Column Major Order.

1. Row Major Order: To find the address of the element using row-major order uses the following formula. Address of $A[i][j] = \text{Base}(A) + w((i-l_r) * N + (j-l_c))$
Example: Find the Address of $\text{arr}[8][6]$ in $\text{arr}[1,10][1,15]$ Base value is 100 in per memory 1 Byte.

Solution: $N = \text{Up} - \text{LB} + 1 = 15 - 1 + 1 = 15$ (Number of column)

$$\begin{aligned} \therefore \text{Address of } [8][6] &= \text{Base} + w((8-1)15 + (6-1)) \\ &= 100 + 1(7 \times 15 + 5) \\ &= 100 + 110 = 210 \end{aligned}$$

2. Column Major order: To find the address using column major order the formula is

$$\text{Address of } A[i][j] = \text{Base}(A) + w((i-l_r) + (j-l_c) \times N)$$

N = Number of rows given in the matrix or size of row = $U_r - L_r + 1$

using before example we can find the column major order is, $N = 10 - 1 + 1 = 10$

$$\begin{aligned} \text{Address } A[8][6] &= \text{Base}(A) + w((8-1) + (6-1)10) \\ &= 100 + 1(7 + 50) \\ &= 100 + 57 = 157 \end{aligned}$$

so the address of column major order is 157.

1(b) Solution

given $A(2:8, -4:1, 6:10, 3:7, -2:6)$

$A[3, -1, 8, 4, L]$

So, $L_1 = 8 - 2 + 1 = 7$

$L_2 = 1 + 4 + 1 = 6$

$L_3 = 10 - 6 + 1 = 5$

$L_4 = 7 - 3 + 1 = 5$

$L_5 = 6 + 2 + 1 = 9$

$E_1 = 3 - 2 = 1$

$E_2 = -1 + 4 = 3$

$E_3 = 8 - 6 = 2$

$E_4 = 4 - 3 = 1$

$E_5 = 2 + 2 = 4$

Row major Order Address is

$$\begin{aligned} &\text{Base}(A) + w(((E_1 L_2 + E_2) L_3 + E_3) L_4 + E_4) L_5 + E_5) \\ \Rightarrow &400 + 4(((1 \times 6 + 3) 5 + 2) 5 + 1) 9 + 4) \\ &= 400 + 4(2128) \\ &= 400 + 8512 = 8912 \end{aligned}$$

Column major Order Address is

$$\begin{aligned} &\text{Base}(A) + w(((E_5 L_4 + E_4) L_3 + E_3) L_2 + E_2) L_1 + E_1) \\ &= 400 + 4(((4 \times 5 + 1) 5 + 2) 6 + 3) 7 + 1) \\ &= 400 + 4(4516) = 400 + 18064 \\ &= 18464 \end{aligned}$$

② Postfix and Prefix notation are alternative ways to represent mathematical expressions, and they have some advantage over infix notation. Hence a few reasons why postfix and prefix notation are used:

1. No need for Operator Precedence and Parenthesis.
2. Easier for Parsing the column.
3. Reduction in Operator Overhead.

Now let's translate the given infix expression $(A * (B + D) / E - F * (G + H / K))$ into postfix notation using stack.

Symbol Stack :

Postfix

(	(	A
A	(	A
*	(*	A
(	(*(	AB
B	(*(	AB
+	(*(+	ABD
D	(*(+	ABD+
)	(*)	ABD+
/	(*/	ABD+E
E	(*/	ABD+E/*
-	(-	ABD+E/*
F	(-	ABD+E/*F
*	(-*	ABD+E/*F
(	(-* (	ABD+E/*F
G	(-* (	ABD+E/*FG
+	(-* (+	ABD+E/*FG
H	(-* (+	ABD+E/*FGH
/	(-* (+/	ABD+E/*FGH
K	(-* (+/	ABD+E/*FGHK
)	(-* +/	ABD+E/*FGHK/
-	(-	ABD+E/*FGHK/+
)	(	ABD+E/*FGHK/+*
)	(	ABD+E/*FGHK/+*-

2) a) How can I minimize overflow in stack given by

First don't allocate big elements on stack like `int big[100000]`; second increase stack size, Thirds in a multi threaded application, may have a relatively small stack by default. Third avoid deep recursion including bugs of infinite recursion. This item is a bit tricky with smart pointers because the destructor of linkedlist-node will call the destructor of the following node, which call the destructor of next node, and so on.

Prefer iterative algorithms over potentially deep recursive algorithms.

2) c) The average case time complexity of the linear search algorithm executed on an array containing n elements is $O(n/2)$ which simplifies to $O(n)$ in Big O notation. In linear search on average you would need to examine half of the elements in the array before finding the desired element (assuming the element is equally likely to be anywhere in the array). This makes the average case time complexity linear or $O(n)$. While the worst case time complexity of linear search is also $O(n)$. It's important to note that the average case analysis assumes that the element you are searching for is equally likely to be at any position in the array. If the distribution of elements in the array is not uniform, the actual average case time complexity may differ.

(a) (i) Formula for find the element of each Array $U-L+1$ (3)
 $\therefore$ Elements for A(5:50) is $50-5+1 = 46$
 Elements " B(1:18) is $18-1+1 = 18$

(ii) Formula for find the address of an element of Array

$$\text{is } A[K] = \text{Base} + W(K-LB)$$

$$\therefore A[35] = 300 + 4(35-5) \\ = 300 + 120 \\ = 420$$

here
 Base(A) = 300
 $W = 4$
 $LB = 5$

$$\text{and } A[55] = 300 + 4(55-5) \\ = 300 + 200 = 500 \text{ (Ans)}$$

3 (b) (i)

0	
1	
2	
3	17
4	
5	19 $\rightarrow$ 26
6	13 $\rightarrow$ 48

$$h(K) = K \bmod 7 \\ \therefore h(19) = 19 \bmod 7 = 5 \\ h(26) = 26 \bmod 7 = 5 \\ h(13) = 13 \bmod 7 = 6 \\ h(48) = 48 \bmod 7 = 6 \\ h(17) = 17 \bmod 7 = 3$$

Hash table for collisions are handled by separate chaining.

(ii)

0	13
1	48
2	
3	17
4	
5	19
6	26

$$h(K) = K \bmod 7 \\ \therefore h(19) = 19 \bmod 7 = 5 \\ h(26) = 26 \bmod 7 = 5 + 1 = 6 \\ h(13) = 13 \bmod 7 = 6 + 1 \bmod 7 = 0 \\ h(48) = 48 \bmod 7 = 6 + 1 \bmod 7 = 0 + 1 = 1 \\ h(17) = 17 \bmod 7 = 3$$

Linear probing Hashing

(iii)

0	
1	48
2	26
3	17
4	
5	19
6	13

Double hashing

$$h(K) = K \bmod 7 \\ h(19) = 5 \\ h(26) = 5 \\ h(13) = 6 \\ h(48) = 6 \\ h(17) = 3$$

$$h'(K) = 5 - (K \bmod 5) \\ h'(26) = 5 - 1 = 4 \\ h_2(26) = h(K) + i \cdot h'(K) \bmod 7 \\ = 5 + 4 \bmod 7 = 2 \\ h'(48) = 5 - (48 \bmod 5) \\ = 5 - 3 = 2 \\ h_2(48) = h(K) + i \cdot h'(K) \bmod 7 \\ = 6 + 2 \bmod 7 \\ = 1$$

3. ① Advantages of using linked storage for storing strings

- 1) Dynamic memory allocation: linked storage allows to dynamic memory allocation, making it easier to manage variable length strings without preallocating fixed sized buffers.
- 2) Efficient memory usage: Linked storage can be more memory-efficient than static array as memory is allocated only as needed, reducing waste.
- 3) Easy string manipulation: Linked lists can simplify string manipulation tasks like inserting or deleting characters, as this can be done without shifting elements.
- 4) Simplified resizing: Adding or removing characters in a linked list can be more straightforward compared to resizing a static array.

Disadvantage of using linked storage for storing strings:

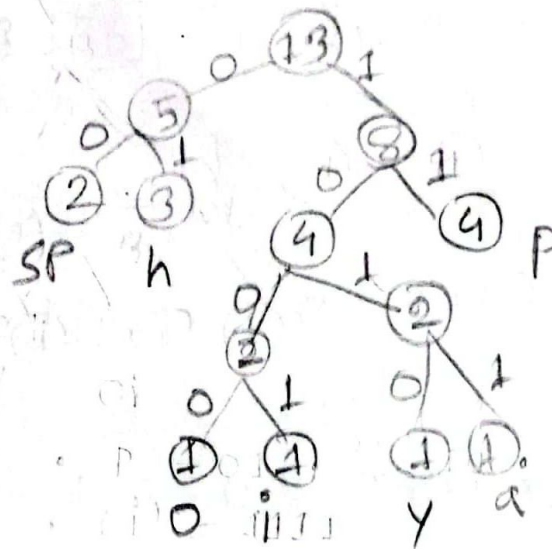
- 1) Increased Overhead: Linked storage adds overhead due to need for pointers, making it less memory-efficient for small strings.
- 2) Slower random access: Accessing characters in a linked list can be slower than array indexing, especially for large strings, due to traversal.
- 3) Fragmented memory: Frequent dynamic memory allocation and deallocation can lead to memory fragmentation and potential inefficiencies.
- 4) Complexity: Managing a linked list of strings can be more complex than using static arrays, requiring additional code for memory management and traversal.

④
 (a) Solution:

given string "Happy hip hop"

character	frequency	code	bits
h	3	01	6
P	4	11	8
sp	2	00	4
a	1	1011	4
y	1	1010	4
i	1	1001	4
o	1	1000	4
			34 bits

using huffman-tree,



Before huffman encoding we need $13 \times 8 = 104$ bits for representing "happy hip hop" but after huffman encoding we need just 34 bits for representing "happy hip hop".

so huffman encoding is $\frac{34}{104} \times 100\%$
 $= 32.69\%$ effective before encoding.

4) b) To find the number of elements in a queue when $FRONT = 2$ and $REAR = 6$ in a circular array of n memory cells, you can use the following formula:

$$\text{Numbers of elements in the queue} = (6 - 2 + 10) \% 10 = 4$$

So, there are 4 elements in the queue.

2. To determine when the array will be filled you need to consider the condition for a full queue in a circular array. A circular queue is considered full when:

$$(REAR + 1) \% n == FRONT$$

in this case, $n = 10$, $FRONT = 2$ and $REAR = 6$

$$\therefore (6 + 1) \% 10 = 7 \% 10 = 7$$

The circular array, considered filled when $(REAR + 1) \% n == FRONT$.

In this case, we find that $(REAR + 1) \% n == 7$

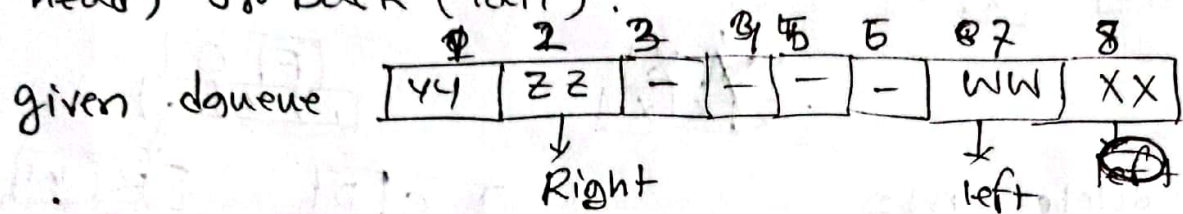
and given that the $FRONT$ is 2

$$\text{so } 7 \neq 2$$

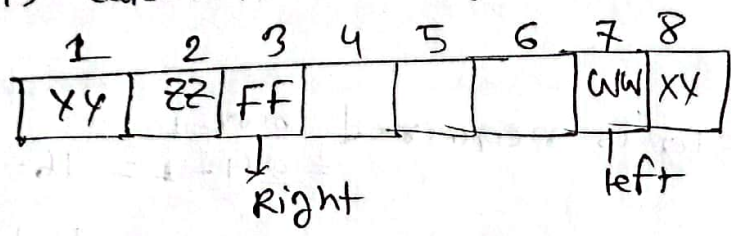
and so it can say that the circular array isn't filled.

Answer to the question no: 05

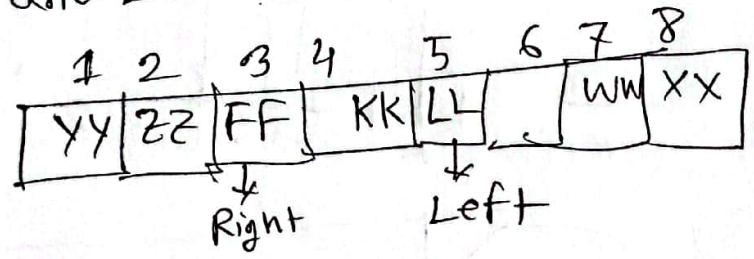
(9) Deque: A deque is a double-ended queue (deque) ^{which} is an abstract data type that generalizes a queue, for which elements can be added to or removed from either the front (head) or back (tail).



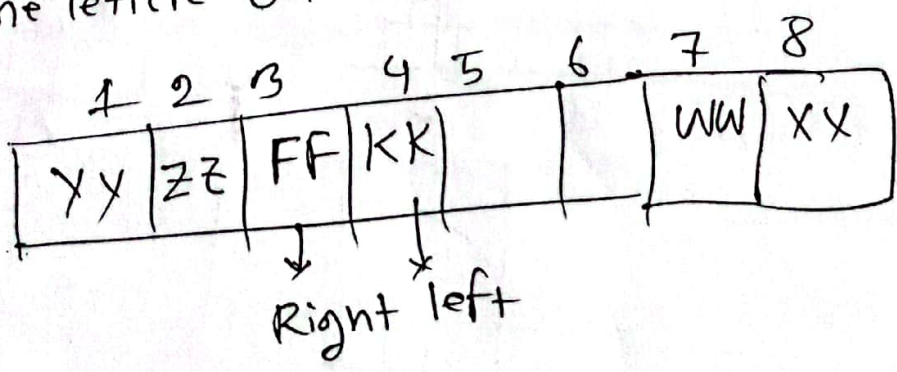
(i) FF is added to the right of deque



(ii) KK and LL are added to the left of the deque

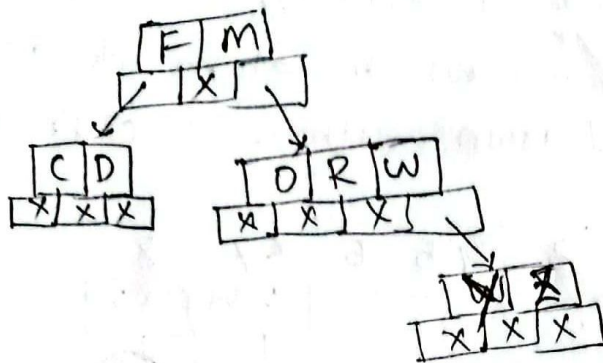


(iii) One letter on the left is deleted

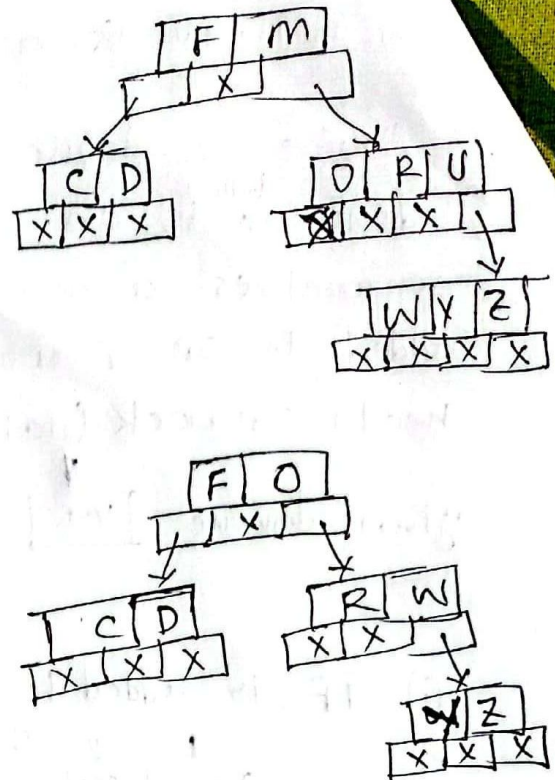


② given 4-way search Tree

After delete U

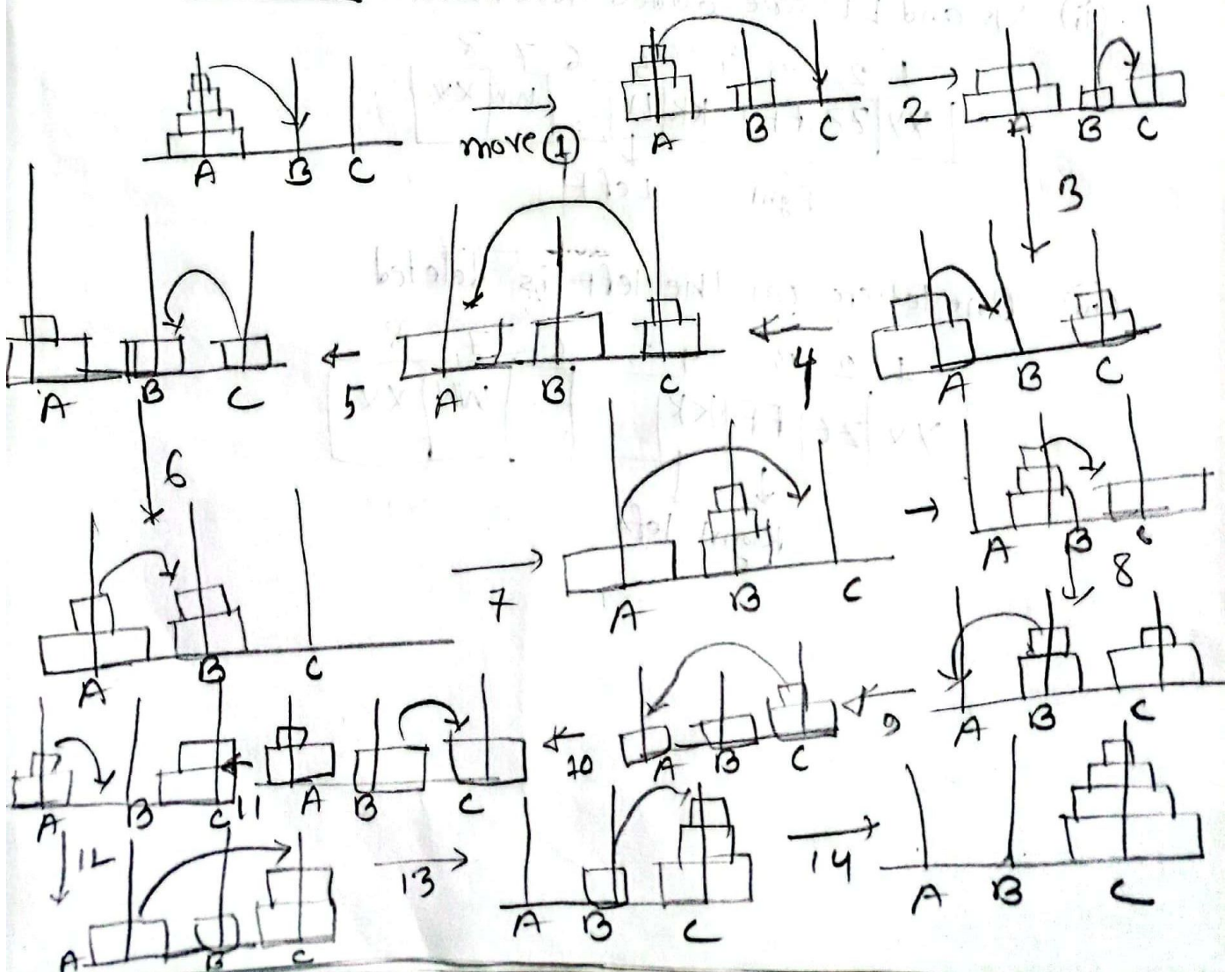


Then delete M:



Step is required $2^n - 1$
 $= 2^4 - 1 = 16 - 1 = 15$

③ Solution: Tower of Hanoi when disk $n=4$



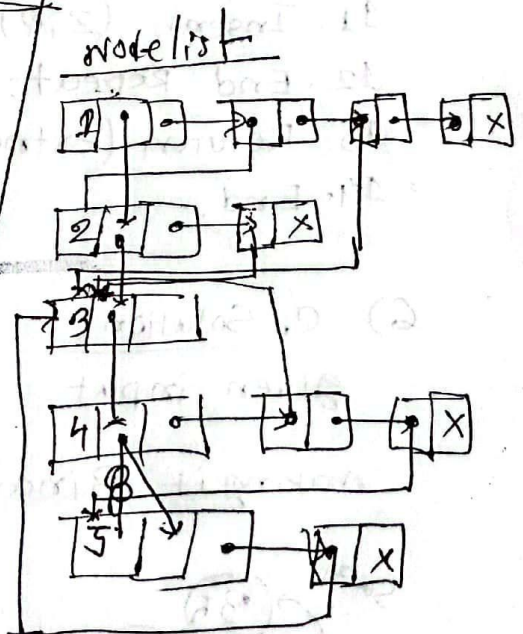
(a) Graph: Graph is a non-linear data structure constituting vertices and edges.

There are two common way to represent the graph.

1. Adjacency matrix.
2. Adjacency linked list.

Linked representation of graph gives the information about the nodes which a specific node is connected. The representation gives the adjacency lists of the vertices with the help of array and linked list.

node	Adj - list
1	2, 3, 4
2	3
3	3, 5
4	3
5	3



linked representation follows → Adjacency list

Array → Adjacency matrix

Adjacency matrix → memory $O(n^2)$

Adjacency List → memory (depends of num edges)

(It can access any node directly)

Adjacency list is fast to add/delete easier than Adjacency matrix.

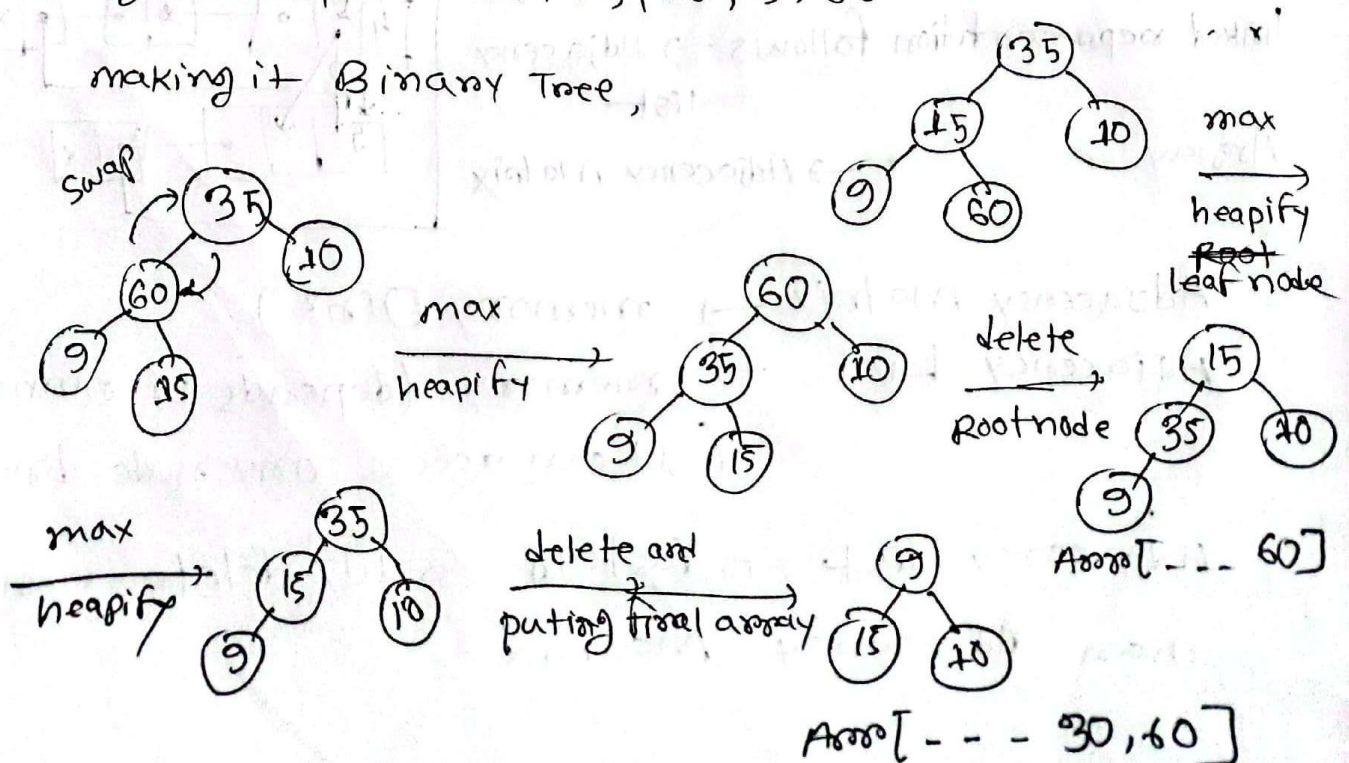
6) ① Solution: Algorithm for Huffman Code

1. Input: Number of message with frequency count.
2. Output: Huffman merge Tree.
3. Begin.
4. Let Q be the priority queue.
5. $Q = \{ \text{initialize priority queue with frequencies of all symbol or message} \}$
6. Repeat $n-1$ times.
7. create a new node z .
8. $\text{extract_min}(Q) = x$.
9. $\text{extract_min}(Q) = y$.
10. $\text{frequency}(z) = \text{frequency}(x) + \text{frequency}(y)$;
11. Insert (z, Q) ;
12. End Repeat.
13. Return $(\text{extract_min}(Q))$
14. End.

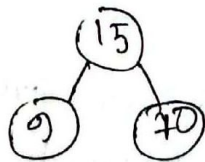
6) ② Solution:

Given input 35, 15, 10, 9, 60

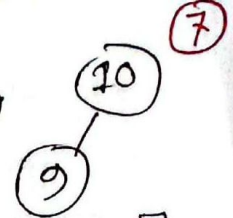
making it Binary Tree,



again max heapify



delete Root and
putting final Array



Array [- - 15, 30, 60]

delete Root
and putting Array



Array [- 20, 15, 30, 60]

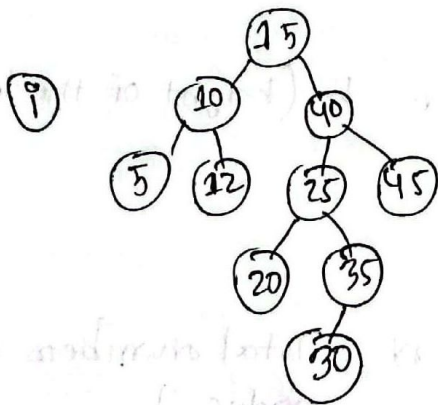
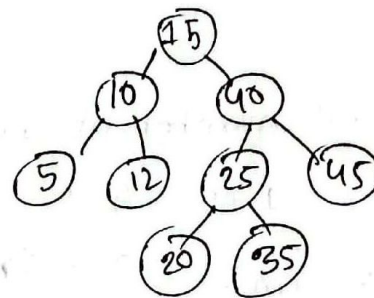
delete
Root

Array [9, 10, 15, 30, 60]

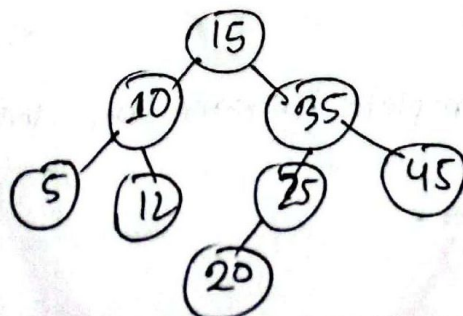
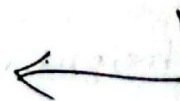
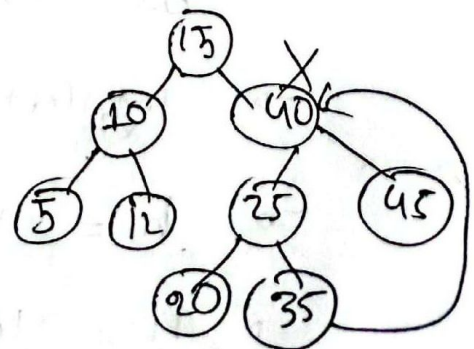
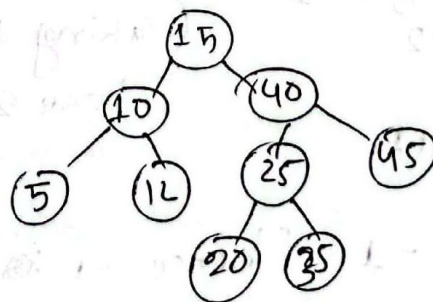
So, Sorted final Array is [9, 10, 15, 30, 60]

Answer to the question NO: 07

(a) soln: given AVL Tree is

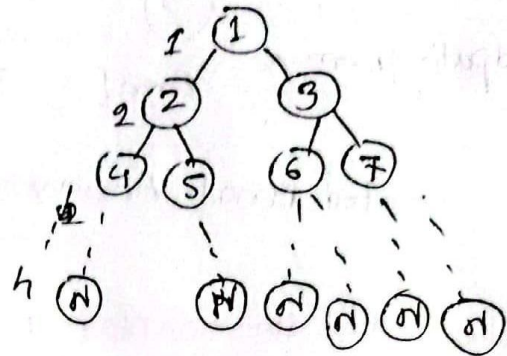


(ii) Remove 40 from the original Tree,



b) Solution:

In a complete binary tree, all levels are completely filled except possibly the last level, which is filled from left to right. The height of a complete binary tree with n nodes can be calculated by determining the number of nodes at each level.



So, total number of nodes in a complete binary tree,
 $1 + 2 + 4 + \dots + 2^h$ (h total levels in tree)

$$\therefore \text{Sum} = \frac{a \times (r^{n+1} - 1)}{r - 1}$$

$$a = 1 ; r = 2/1 = 2 ; n = h \text{ (height of the tree)}$$

$$\therefore \text{Sum} = \frac{1 \times (2^{h+1} - 1)}{2 - 1}$$

$$\Rightarrow N = 2^{h+1} - 1 \quad (N = \text{Total number of nodes})$$

$$\Rightarrow N + 1 = 2^{h+1}$$

$$\Rightarrow \log_2(N+1) = \log_2 2^{h+1} \quad (\text{Taking the logarithm base 2 on both side})$$

$$\Rightarrow \log_2(N+1) = h+1$$

$$\Rightarrow h = \log_2(N+1) - 1 \quad [\text{hence } -1 \leq h \leq N]$$

$$\Rightarrow h = \log_2 N$$

$\therefore$ So the height of a complete binary tree with n nodes is approximately $\log_2(n)$.

7. (C) Answer: The time complexity of algorithms is 8 generally expressed using big-O notation which describe the upper bound on the growth rate of the algorithm with respect to the input size. Let's compare the time complexity of linear search (T_1) and binary search (T_2) for two different values of $n = 1000$ and $n = 10000$.

1. Linear Search (T_1):

1. In the worst case, linear search has a time complexity of $O(n)$ as it needs to potentially check each element in the list until the target is found.

2. Binary search (T_2):

• Binary search has a time complexity of $O(\log n)$ in the worst case, as it repeatedly divides the search space in half.

Now, let's compare the time complexity for $n = 1000$ and $n = 10000$.

i) For $n = 1000$

- Linear Search: $T_1 = O(1000) = O(n)$
- Binary Search: $T_2 = O(\log 1000) = O(\log n)$

ii) For $n = 10000$

- Linear Search: $T_1 = O(10000) = O(n)$
- Binary Search: $T_2 = O(\log 10000) = O(\log n)$

In both cases, the time complexity of binary search (T_2) is significantly better than the time complexity of linear search (T_1) as the size of dataset (n) increases.

The logarithmic time complexity of binary search allows it to scale much more efficiently for large datasets compared to the linear time complexity of linear search.

Answer to the question NO: 08

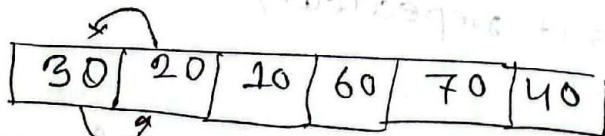
(a) Sorting: Sorting is the process of arranging elements in a specific order, often in ascending or descending order based on a certain key.

Types of sorting Algorithm.

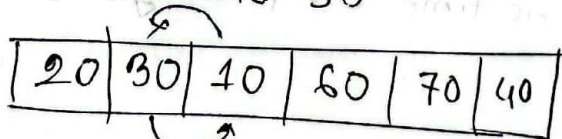
1. Bubble Sort. 2. Selection Sort. 3. Insertion Sort
4. Merge Sort. 5. Quick Sort. 6. heap Sort.
7. Radix Sort. 8. Counting Sort.

(b) Solution: insertion Sort:

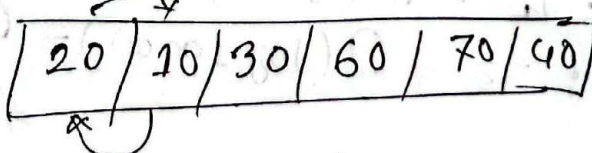
given array [30, 20, 10, 60, 70, 40]



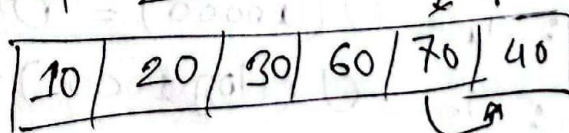
Swap 20 and 30



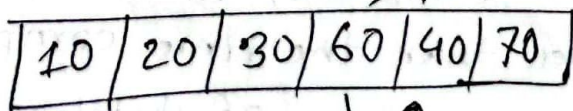
Swap 30 and 10



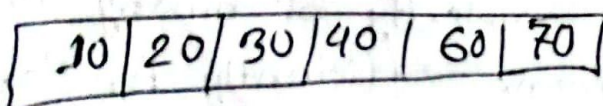
Swap 20 and 10



Swap 40 and 70



Swap 40 and 60



→ This is the sorted Array.

Bubble Sort :

9

Initial state: [30, 20, 10, 60, 70, 40]

Pass 1:

20, 30, 10, 60, 70, 40
20, 10, 30, 60, 70, 40
20, 40, 30, 60, 70, 40
20, 10, 30, 60, 70, 40
20, 40, 30, 60, 40, 70

Pass 2:

10, 20, 30, 60, 40, 70
10, 20, 30, 60, 40, 70
10, 20, 30, 60, 40, 70
10, 20, 30, 40, 60, 70

There is no more swap needed for this the
Sorted array is 10, 20, 30, 40, 60, 70

2) Solution:

19

Algorithm of Selection sort.

Selection Sort (arr, n)

Step 1: Start.

Step 2: Repeat steps 2 and 3 for $i = 0$ to $n-1$.

Step 3: call smallest (arr, i, n, pos)

Step 4: swap $arr[i]$ with $arr[pos]$

[End of Loop]

Step 5: Exit.

Smallest (arr, i, n, pos)

Step 1: [initialize] set $small = arr[i]$

Step 2: [initialize] set $pos = i$

Step 3: Repeat for $j = i+1$ to n

if ($small > arr[j]$)

set $small = arr[j]$

set $pos = j$

[End of if]

[End of Loop]

Step 4: Return pos.

Time Complexity of selection sort Algorithm.

(i) For Best case $O(n^2)$

(ii) For Average case $O(n^2)$

(iii) For worst case $O(n^2)$

Drawbacks of selection Sort:

1. Inefficiency for Large Datasets:

The quadratic time complexity makes selection sort inefficient for large database. It becomes impractical when dealing with a large number of elements.

2. Unstable Sort.
3. No early termination.
4. Not Adaptive.
5. Not stable sort.

In Summary, while selection sort is simple to understand and implement, its inefficiency and lack of stability make it less suitable for practical use, especially for large datasets.

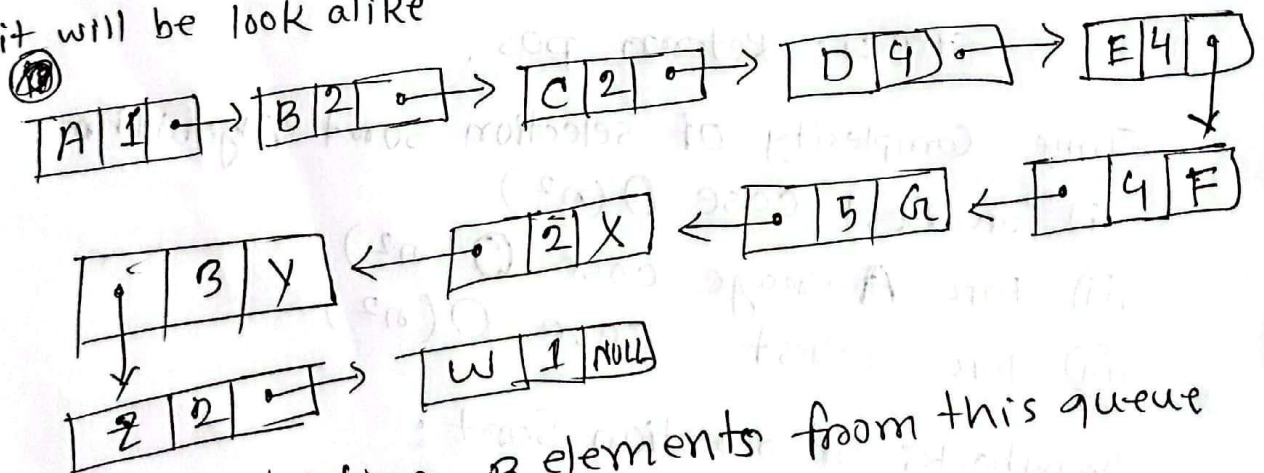
2) (b) solution:

(i)

	INFO	PRN	LINK
1	B	2	6
2	X	2	7
3	D	4	4
4	E	4	9
5	A	1	1
6	C	2	3
7	Y	3	10
8	G	5	2
9	Q	4	8
10	F	2	11
11	Z	1	12
12	W		0

START →
[5]

it will be look alike



(ii) After deleting 3 elements from this queue

